Desarrollo de un chat cifrado a través de RSA, así como las funciones para cifrar y descifrar RSA y un sistema de registro de usuarios

Pablo Tuñón Laguna Jorge Vančo Sampedro

Índice

Introducción 3
Funciones de RSA 4
Generación de claves 4
Lista de primos de cifras grandes 4
Agregación y eliminación de padding 6
Máximo padding 6
Cifrado y descifrado a través de RSA 7
Cifrado de ASCII con RSA 8
Descifrado a ASCII 9
Descifrado a cadena 9
Cifrar mensaje a cadena 10
Descifrar mensaje a cadena 11
Ruptura de clave 11
Ataque de texto elegido 12
Ruptura del RSA desarrollado con anterioridad 13
Ficheros complementarios 14
Registrar usuario 14
Criptochat 14

Introducción

El RSA, que recibe su nombre de las siglas de los desarrolladores (Rivest, Shamir y Adleman) es uno de los algoritmos de cifrado más usado hoy día. RSA se basa en algoritmos y leyes de la aritmética modular, por ello para el desarrollo de las funciones de cifrado se va a hacer uso de la librería de funciones y algoritmos modulares desarrollada anteriormente: “modular.py”.

El RSA es uno de los cifrados más robustos de nuestros tiempos superado únicamente por algoritmos cuánticos y de curva elíptica. Debido a ello es muy habitual encontrarlo en cifrados de objetos importantes diarios. El DNI electrónico por ejemplo está cifrado con RSA de 2048 bits, el RSA desarrollado a lo largo de este escrito puede cifrar y descifrar con claves de 192 bits en tiempos cortos, aparte se ha desarrollado una función que se explicará posteriormente capaz de romper RSA con claves algo menores.

Con el objetivo de entender mejor el cifrado de RSA se han desarrollado dos programas complementarios. Un primer programa que se encarga de registrar la clave pública y privada de un usuario, en base de 2 números menores de 25 cifras introducidos por el usuario y un número máximo de cifras de padding. El segundo programa ejemplifica RSA permitiendo el cifrado y descifrado de textos fruto de la comunicación entre dos usuarios, la explicación de los mismos se encuentra en las páginas posteriores a la explicación de las funciones de RSA y que se basan en las mismas.

Cabe destacar que para restarle trabajo al lector se aplica la siguiente convección, todo algoritmo y proceso que se encuentre desarrollado en la librería de algoritmos “modular.py” al final se indicará (*). De tal manera que se sabe que el algoritmo de cipolla(*) se encuentra implementado en dicha librería.

Funciones de RSA

A continuación, se van a explicar las funciones implementadas para realizar el algoritmo de RSA.

Generación de claves

RSA es un algoritmo de cifrado asimétrico, ya que usa una clave para cifrar y otra para distinta para descifrar, a lo largo del trabajo se explicará el funcionamiento de las claves. El proceso de obtención de claves es el siguiente, lo primero que hay que hacer es tomar una pareja de números primos (distintos) *p* y *q*, dichos números primos se multiplicarán para obtener *n*. Posteriormente se calcula la phi de Euler(*) sobre n obteniendo *phi_n* a continuación se busca un número *e* tal que este sea coprimo con *phi_n* y además sea menor que *phi_n*. Al obtener *e* se calcula *d* como la inversa de *e* módulo *n*. Por ello se cumple lo siguiente:

$$ed \equiv 1 mod(\varphi(n))$$

Los números *e* y *d* son la clave pública y privada (respectivas) del usuario, posteriormente se usará esta misma notación numérica para explicar el cifrado y descifrado de la clave. A continuación, se va a explicar cómo se ha realizado este proceso en código.

En primer lugar se ha obtenido una lista de primos entre dos números(*) ya que el programa de registrar usuario, para mayor seguridad no permite la elección de primos sino que pide al usuario un rango en el que se pueden encontrar los primos. En caso de que el rango incluya números de grandes cifras, se usa una función para generar la lista de primos con números grandes:

Lista de primos de cifras grandes
Esta función busca números primos en un rango específico examinando valores desde el principio hacia arriba, desde el medio hacia arriba y hacia abajo, y desde el final hacia abajo. Utiliza la optimización de que solo se consideran números primos aquellos que son congruentes con 1 (1 mod 6) o 5 (5 mod 6) para acelerar la búsqueda.

La función surge precisamente en respuesta a un problema que la función de generación de primos original plantea, puede quedarse atascada o fallar buscando primos en intervalos muy grandes. Dado que ahora mismo no se necesitan todos los números primos en ese rango, la función calcula

2000 primos y para, otros casos en los que para es si han pasado 10 segundos y ya posee dos primos distintos o si han pasado 15 y no ha encontrado ningún primo.

Obtenida ya una lista de primos, se verifica que sea correcta, es decir, en caso de que la lista contenga más de un elemento, la lista es correcta debido a que esta contiene al menos dos primos distintos en el rango de valores indicados por el usuario. Posteriormente se escogen dos números primos aleatorios y distintos de la lista y se multiplican para obtener *n*. A continuación se calcula la phi de Euler(*) de *n* y se escoge la e como un número aleatorio coprimo mayor que *phi_n*//2 y menor que *phin_n*. El paso final consiste en obtener d, obtenida como el inverso multiplicativo(*) de *e* módulo *n*, tal y como se puede observar a continuación.

$$d = e^{-1} mod(n)$$

A continuación, se muestra el código implementado:

```
if min_primo > max_primo:
    min_primo, max_primo = max_primo, min_primo

try:
    prime_list = modular.lista_primos(min_primo, max_primo)

except OverflowError:
    prime_list = lista_primos_muy_grandes(min_primo, max_primo)

except MemoryError:
    prime_list = lista_primos_muy_grandes(min_primo, max_primo)

if (prime_list == None or len(prime_list) <= 1):  # Eliminación de casos en los que sólo quedaría un elemento o 0 elementos en la lista
    return None, None, None

length = len(prime_list)

p = prime_list[rd.randint(0, length - 1)]
q = prime_list[rd.randint(0, length - 1)]

while p == q:
    q = prime_list[rd.randint(0, length - 1)]

n = p * q
phi_n = n - p - q + 1  # (p - 1) * (q - 1)
e = None

while not e or not modular.coprimos(e, phi_n):
    e = rd.randint(phi_n // 2, phi_n - 1)

d = modular.inversa_mod_p(e, phi_n)  # Cálculo de d

return (n, int(e), int(d))
```

Figura 1: implementación de generación de claves

Agregación y eliminación de padding

El padding es un relleno usado por la criptografía de tal forma que el cifrado sea más robusto a ataques a través de criptoanálisis. Funciona de la siguiente manera, el texto plano o mensaje a transmitir se segmenta en fragmentos y estos (en función del tipo de padding) se completan a veces por delante y por detrás otras veces por detrás del fragmento hasta llegar a una longitud adecuada o hasta que se puede aplicar una función concreta.

En el caso del desarrollo de este RSA la función de padding usada añade un número de cifras aleatorias al final del mensaje. El padding posible se calcula mediante la siguiente función.

Máximo padding

Calcula el padding máximo que se puede usar según el módulo de la clave pública (n). Al cifrar usando el valor Unicode de los caracteres, se usa su valor máximo, el 1114112 como umbral para evitar colisiones. Además, se le suma 1 a este umbral para que este valor pueda ser congruente con otro módulo n al aplicarle padding.

```
maximun = max(int(math.log10(n / 1114112 + 1)), 0)
return maximun
```

Figura 2: implementación de máximo padding

Realmente esta función se usa en el registro de usuario, ya que a la función de aplicar el padding se le introduce el padding necesario, pero dado que en esta sección se explica el uso del padding y su aplicación se ha considerado relevante realizar su explicación en esta parte del escrito.

Antes de aplicar padding se comprueban los casos límites(a fin de que no se interrumpa el programa) devolviendo el mismo mensaje si las cifras de padding son nulas y en caso de que las cifras de padding sean negativas, se convierten a positivas. Finalmente se aplica el padding de la siguiente manera, se multiplica el mensaje original por 10 elevado al número de cifras de padding correspondientes y se le suman tantas cifras de padding aleatorias como se haya indicado(ver Figura 3).

```
if digitos_padding == 0:
    return m

if digitos_padding < 0:
    digitos_padding = -digitos_padding

padding = rd.randint(10 ** (digitos_padding - 1), 10 ** (digitos_padding) - 1)
if m < 0:
    padding = -padding
return m * 10**digitos_padding + padding
```

Figura 3: implementación de aplicación de padding sobre un mensaje

Eliminar el padding es un proceso sencillo, basta con realizar la división entera entre el mensaje con padding y 10 elevado al número de padding que posee el mensaje(ver Figura 4)

```
if m == None:
    return None


return m // (10**digitos_padding)
```

Figura 4: Implementación de eliminar el padding de un mensaje

Cifrado y descifrado a través de RSA

El cifrado de RSA se realiza de la siguiente manera: a un mensaje (escrito como un entero de menor número de cifras que *n*) se eleva a la clave pública *e* y al resultado se le aplica el módulo de *n*, tal y como se puede observar a continuación:

$$texto\ cifrado\ = (texto\ plano)^e mod(n)$$

A la hora de descifrar basta con elevar el texto cifrado a la clave privada *d* y realizar módulo *n* de la siguiente manera:

$$texto\ plano = (texto\ cifrado)^d mod(n)$$

El descifrado funciona debido a que *e* y *d* son coprimos, por tanto se pueden aplicar el pequeño teorema de Fermat:

$$1 = e \times d\ mod(\varphi(n))$$

Por el pequeño teorema de Fermat se puede descifrar de la siguiente manera:

$$(texto\ cifrado)^d mod(n) = (texto\ plano)^{e \times d} mod(n) = (texto\ plano)^1 mod(n)$$

Permitiendo así el descifrado haciendo uso de la clave pública *e* y la clave privada *d*.

Para esta funcionalidad se han generado un total de 4 funciones, dos que descifran y cifran en RSA cuando reciben un valor ASCII y otras dos que se valen de las anteriores que reciben una cadena de texto y una lista de enteros y cifran o descifran, a continuación se dan descripciones de la implementación de dichas funciones.

Cifrado de ASCII con RSA

En primer lugar, se eliminan los casos en los que el cifrado puede fallar, si el mensaje es igual a None, si el mensaje es negativo, si el módulo es 0 y si la clave pública es 0, devolviendo en todos los casos None. A continuación, se sigue trabajando con los casos límite, haciendo el módulo, la *e* y los dígitos de padding positivos siempre. El último caso límite evaluado es aquel en el que los dígitos de padding son superiores a los dígitos del módulo, que también se devuelve None.

Finalmente sólo resta aplicar el padding al mensaje con los correspondientes dígitos de pading y hacer la potencia módulo *n*(*) del texto cifrado.

```
if m == None or m < 0:  # Casos en los que el mensaje falla
    return None

if (n == 0 or e == 0):  # Comprobación de que módulos, exponente y los dígitos de padding no son 0 o negativos
    return None

if n < 0:
    n = -n

if e < 0:
    e = -e

if digitos_padding < 0:
    digitos_padding = -digitos_padding

if digitos_padding > len(str(n)):  # El padding no puede ser mayor que el módulo o habría error en el cifrado
    return None

m_padding = aplicar_padding(m, digitos_padding)  # Aplicación de padding
cipher_text = modular.potencia_mod_p(m_padding, e, n)  # Cifrar como m^e mod(n)

return cipher_text
```

Figura 5: implementación de cifrado RSA cuando el mensaje es un código ASCII

Cifrado de cadena con RSA

En primer lugar se usa una función que convierte la cadena en una lista de strings en la que cada valor es un carácter (ver Figura 6) y posteriormente obtiene su valor ASCII devolviendo una lista con los mismos.

```
if type(s) != str or len(s) == 0:  # Verifica que la cadena sea un dato correcto
    return None

return [ord(letter) for letter in s]  # Creación de lista de ascii
```

Figura 6: implementación de convertir string en lista de números ascii, una por cada carácter

Posteriormente, en la función de cifrado, se comprueba que el resultado sea correcto (es decir que el tipo del mensaje a cifrar era una string de longitud mayor a 0) y mediante un bucle for cifra cada uno de las posiciones de la lista usando la función cifrado de ASCII con RSA

```
ascii_message = codificar_cadena(s)
if ascii_message is None:
    return None

ciphered_text = []

for ascii_number in ascii_message:
    if ascii_number is None:
        return None

    else:
        ciphered_text.append(cifrar_rsa(ascii_number, n, e, digitos_padding))

return ciphered_text
```

Figura 7: implementación de convertir string a lista de enteros cifrados

Descifrado a ASCII

A la hora de descifrar, nuevamente se comienza eliminando los casos límite, en caso de que *n* sea igual a 0, *d* sea igual a 0, el mensaje sea None o negativo se devuelve None. También se devuelve None si los dígitos de padding son mayores a *n* como en el caso del cifrado. Para terminar el análisis de casos límite, se calcula el valor absoluto de *n*, *d* y el número de dígitos de padding y se iguala a las variables anteriores, de tal manera que se trabaje con números naturales.

Finalmente se realiza la potencia módulo *n*(*) del mensaje cifrado elevado *e* y eliminar el padding del texto obtenido, de esta manera se obtiene un valor ASCII perteneciente al mensaje plano a partir de un entero cifrado.

```
if (n == 0 or d == 0 or c is None or c < 0):  # Comprobación de que módulos, exponente son cero
    return None

if n < 0:
    n = -n

if d < 0:
    d = -d

if digitos_padding < 0:
    digitos_padding = -digitos_padding

if digitos_padding > len(str(n)):  # El padding no puede ser mayor que el módulo o habría error en el descifrado
    return None

m_plain = modular.potencia_mod_p(c, d, n)  # Descifrado como c^d mod(n)
m_plain = eliminar_padding(m_plain, digitos_padding)  # Eliminación de padding

return m_plain
```

Figura 8: implementación de descifrado de RSA cuando el texto cifrado es un entero

Descifrado a cadena

La función realiza el cifrado a la inversa, de tal manera que recibe una lista de enteros cifrados, descifra la lista a una lista de valores ASCII a través de la función anterior y posteriormente son recogidos por otra función que los convierte en cadena de texto. En el caso de la función que pasa de lista de enteros cifrados a lista de valores ASCII se evalúa que ningún entero sea None ya que

el resto de excepciones las maneja la función de descifrado a partir de enteros, posteriormente se aplica dicha función(ver Figura 8) sobre cada uno de los valores de la lista de enteros cifrados

```
if len(cList) == 0:
    return None

deciphered_text = []

for c in cList:
    if c is None:
        return None

    else:
        deciphered_text.append(descifrar_rsa(c, n, d, digitos_padding))

try:
    decoded_text = decodificar_cadena(deciphered_text)

except ValueError as e:
    raise ValueError(e)

return decoded_text
```

Figura 9: implementación de descifrado RSA de lista de enteros cifrados a lista de valores ASCII

Finalmente, resta aplicar la función que convierte los valores ASCII a una única string, que evalúa que la longitud del código ASCII sea mayor que 0, que el valor introducido sea de tipo entero y que el valor ASCII sea mayor que 0 y menor que 1114112 ya que es el mayor valor de encoding Unicode posible. Comprobados dichos casos, transforma cada uno de los caracteres ASCII a strings y las une en una única (ver Figura 10).

```
if len(m) == 0:
    return None

decodified_string = ""

for number in m:
    # Evaluación de casos en los que chr daría error
    if type(number) != int or number < 0 or number > 1114112:
        raise ValueError("Algún entero no representa un caracter unicode válido")

    else:
        decodified_string += chr(number)  # Conversión de ascii a string

return decodified_string
```

Figura 10: implementación de paso de lista de valores ASCII a una única cadena

Aparte, también se implementaron dos funciones que luego simplifican el guardado y acceso a datos tanto para cifrar como para descifrar.

Cifrar mensaje a cadena

Se cifra un mensaje haciendo uso de la función de la Figura 7 y se unen los resultados en una cadena, separando los enteros cifrados por espacios.

```
return " ".join(str(char) for char in cifrar_cadena_rsa(s, n, e, digitos_padding))
```

Figura 11: implementación de cifrar mensaje a cadena de enteros separados por espacios

Descifrar mensaje a cadena
Recibe una cadena de enteros cifrados de acuerdo a la función anterior, los convierte en enteros y los convierte a una cadena de caracteres haciendo uso de la función de la Figura 8.

```
cList = [int(num) for num in c.split(" ")]

try:
    return descifrar_cadena_rsa(cList, n, d, digitos_padding)

except ValueError as e:
    raise ValueError(e)
```

Figura 12: Implementación de descifrar un mensaje a cadena de texto de cadena de enteros separados por espacios

Ruptura de clave

Tal y como se comentó anteriormente, debido que en la librería "modular.py" se desarrolló una eficiente forma de factorización a través de rho de pollard y la criba de Eratóstenes mejorada, se pueden romper claves de un número de cifras considerables en tiempos aceptables. La base de romper una clave radica en factorizar la n, en el caso de un RSA auténtico, esto es complejo tedioso y prácticamente imposible, debido a que los *p* y *q* son de más de 100 cifras.

El fundamento matemático reside en el hecho de que *p* y *q* son primos, por lo que al multiplicarlos, sólo existe una forma de factorizarlo, una vez hallada la factorización es trivial hallar *e*, *d* y *phi_n*. Es más a nivel de código es tan simple como calcular la factorización de n, y una vez que se tiene *p* y *q* se calculan el resto de términos como en la Figura 1.

```
factores = modular.factorizar(n)
factores_exponentes = factores.items()
phi_n = 1

for f, exp in factores_exponentes:
    phi_n *= f ** (exp - 1) * (f - 1)

try:
    d = modular.inversa_mod_p(e, phi_n)

except ValueError:
    raise ValueError()

return d
```

Figura 13: Implementación de romper la clave de un RSA y obtener la clave privada

Ataque de texto elegido

No se le puede atribuir a un único autor este ataque, a pesar de ello es bastante conocido por su sencillez en caso de que el RSA no tenga padding. El ataque consiste en crear un diccionario cifrado(ver Figura 14) en el que cada carácter(en este caso los del unicode) está asociado a una clave, que corresponde a cifrar ese carácter con padding 0, la *n y* la *e*, de tal forma que basta con buscar coincidencias entre las claves del diccionario y el texto cifrado y cuando se encuentra una se devuelve el valor asociado a esta clave(ver Figura 15).

```
cifrado = dict()
max_unicode = 1114112

for unicode in range(0, max_unicode):
    cifrado[cifrar_rsa(unicode, n, e, padding_digits)] = chr(unicode)

return cifrado
```

Figura 14: Implementación de diccionario de caracteres cifrados de clave y caracteres descifrados de valor

```
m = ""
cifrado = generar_diccionario(n, e, padding_digits)
for c in cList:
    try:
        m += cifrado[c]

    except KeyError:
        raise ValueError(
            "El mensaje no se corresponde con ningún texto plano que haya sido codificado con RSA sin padding."
        )

return m
```

Figura 15: Implementación de ataque de texto elegido usando diccionario de Unicode cifrado.

Cabe destacar que en caso de que se conozca el padding se puede realizar el mismo ataque salvo que el diccionario en lugar de estar cifrado con el padding 0, estará cifrado con el padding del usuario. En el desarrollo de los códigos, se recibió un mensaje cifrado de un usuario X con padding 0, este ha sido descifrado haciendo uso del ataque de texto cifrado y se encuentra a continuación:

“Con diez cañones por banda,// viento en popa a toda vela,// no corta el mar, sino vuela// un velero bergantín;// bajel pirata que llaman,// por su bravura, el Temido,// en todo mar conocido// del uno al otro confín.// La luna en el mar riela,// en la lona gime el viento// y alza en blando movimiento// olas de plata y azul;// y va el capitán pirata,// cantando alegre en la popa,// Asia a un lado, al

otro Europa,// y allá a su frente Estambul;”

Ruptura del RSA desarrollado con anterioridad

Debido a la forma en la que se ha implementado el RSA anterior, no es muy seguro, en primer lugar por la longitud de las claves que puede manejar, que son fáciles de factorizar, en segundo lugar porque el padding determinista lo hace sensible a el ataque que se va a describir a continuación.

Se rompe la *n* factorizándola(*) y se prueba a descifrar el texto cifrado con la clave privada empezando con padding 0 y finalizando con el máximo padding correspondiente a la *n* calculado con la función de la Figura 2. Todos los resultados que sean descifrables (ya que hay casos que no son posibles) se imprimen por pantalla y el usuario puede escoger cuál considera que es el texto original, conociendo además el padding usado y la clave privada del emisor del mensaje.

```
d = romper_clave(n, e)
print(f"A continuación se muestra la n: {n}, la e:{e} y la d:{d} usadas para cifrar el mensaje a descifrar.")

max_padding = max_theoretical_padding(n)
posibles_m = []

for padding in range(0, max_padding + 1):
    try:
        m = descifrar_mensaje_rsa(c, n, d, padding)
        print(m)
        posibles_m.append(m)

    except ValueError:
        print("No puede ser con padding", padding)

return posibles_m
```

Figura 16: Implementación de ataque a través de ruptura de clave y pruebas de padding.

Ficheros complementarios

A continuación, se va a realizar una breve explicación de los dos ficheros generados para poner en práctica el RSA desarrollado

Registrar usuario

Este fichero permite la creación de un usuario, con su correspondiente clave pública y clave privada, estas se guardarán como ficheros de texto en el directorio “Usuarios” que en caso de no existir se creará. El fichero hace uso de las funciones desarrolladas en la librería RSA y pide un nombre para el usuario, un valor mínimo para buscar primos, un valor máximo y posteriormente calcula el padding máximo y le pide al usuario que introduzca un número de padding mayor que 0 y menor o igual que ese número para guardar este en el fichero de clave pública y que otros usuarios se puedan comunicar con él.

Criptochat

El fichero permite simular la comunicación entre dos usuarios, cuya información debe existir en el directorio de usuarios ya que si no, no podrá ejecutar los cifrados y descifrados. Una vez que existe un usuario emisor y un usuario receptor criptochat permite al usuario 1 o emisor cifrar un mensaje para el usuario 2 o receptor, también permite descifrar un mensaje que le haya mandado el usuario 2. El cifrado se puede realizar únicamente introduciendo un texto por teclado sin embargo el descifrado se puede hacer a través de un texto por teclado y a través de un fichero de información cifrada del directorio de Mensajes. Por ello, tal y como cabe esperar se ha ideado un sistema de guardado (que permite elegir al usuario si quiere guardar o no) que guarda los ficheros cifrados y permite a otros usuarios descifrarlos en caso de disponer de la clave pública, sin sobrescribir nunca un mensaje anterior.